

AMS/OSRAM AS5048A magnetic rotary encoder

Internal notes – SPI driver, wiring, calibration

Repo `Damiao_test`

May 12, 2026

1 What it is

The AS5048A is a contactless 360° magnetic rotary position sensor manufactured by AMS / ams OSRAM Group. A lateral Hall array on the chip measures the field of a small diametrically magnetised disc glued to the rotating shaft above the package; an internal CORDIC computes the angle and an SPI interface returns it as a 14-bit value.

Property	Value
Resolution	14 bit (16384 counts/rev, 0.0219°/LSB)
Sampling rate	11.25 kHz typ.
Propagation delay	100 µs typ.
SPI clock	up to 10 MHz (mode 1, CPOL= 0, CPHA= 1)
Frame length	16 bit, MSB first, even parity
Supply	3.0–3.6 V (VDD3V) or 4.5–5.5 V (VDD5V via LDO)
Operating temp.	–40 to +150°C (5 V) / +125°C (3.3 V)
Package	TSSOP-14 (5 × 6.4 mm)
Magnet	diametrically magnetised, 6–8 mm, 30–70 mT at die

Variants. **AS5048A** = SPI + PWM, **AS5048B** = I²C + PWM (same die, different output pins). This repo uses the A part on an AMS evaluation/adaptor board, connected via SPI to a Jetson.

2 Wiring: encoder → Jetson Orin Nano

The encoder adapter board exposes six pins; map them directly to the SPI0 pins on the Orin Nano 40-pin header:

Encoder pin	Jetson pin	Jetson signal
GND	25	GND
3V3	17	3.3 V
MOSI	19	SPI0_MOSI
MISO	21	SPI0_MISO
SCK	23	SPI0_SCK
CSN	24	SPI0_CS0

This corresponds to the device node `/dev/spidev0.0` (verified by the loopback test; see Section 4.2). The SPI pins are tristated by default on every boot and must be activated via a device tree overlay – see Section 3.

Powering. The adapter board’s on-board LDO accepts both 3.3 V and 5 V, but the Jetson header is 3.3 V logic only – always power from pin 17. Do *not* connect 5 V to a board whose pull-ups are referenced to 3.3 V.

3 Enabling SPI on the Jetson: device tree overlay

The SPI pads on the 40-pin header are **tristated (high-Z) by default** after every boot. They must be mux'd to the SPI controller with a pinmux overlay before the kernel driver will accept transfers.

3.1 Option A – jetson-io.py (interactive, not used here)

Run `sudo /opt/nvidia/jetson-io/jetson-io.py`, select “Configure Jetson 40pin Header”, toggle “SPI1”, save and reboot. This writes a pre-baked DTBO supplied by NVIDIA. It works for a quick test but the overlay is untracked and cannot be customised.

3.2 Option B – custom DTBO (used in this repo)

A hand-authored overlay lives at:

`jetson-orin-spi-overlay-guide/examples/as5048a-encoder/`

Files in that folder:

- `jetson-orin-as5048a.dts` – DTS source. Configures the four Tegra pads `spi1_mosi_pz5` / `spi1_miso_pz4` / `spi1_sck_pz3` / `spi1_cs0_pz6` (header pins 19/21/23/24) to function `spi1`. No extra GPIO fragment is needed because the encoder's CSN is driven by hardware CS0.
- `install.sh` – compiles the DTS with `dtc`, copies the DTBO to `/boot/`, backs up `/boot/extlinux/extlinux.conf` with a timestamp, appends a new LABEL `AS5048A` entry, and sets `DEFAULT AS5048A`.

Installation:

```
1 cd jetson-orin-spi-overlay-guide/examples/as5048a-encoder
2 chmod +x install.sh
3 sudo ./install.sh # compiles DTS, installs DTBO, updates extlinux.conf
4 # reboot when prompted
```

What extlinux.conf looks like after the installer runs:

```
1 DEFAULT AS5048A
2
3 LABEL primary
4     LINUX /boot/Image
5     APPEND ...
6
7 LABEL AS5048A
8     LINUX /boot/Image
9     FDT /boot/dtb/kernel_tegra234-p3768-0000+p3767-0005-nv-super.dtb
10    INITRD /boot/initrd
11    APPEND ...
12    OVERLAYS /boot/jetson-orin-as5048a.dtbo
```

U-Boot reads this file at boot, loads the base DTB, patches it with the overlay before handing it to the kernel, and the four SPI pads come up mux'd. Confirm pinmux was applied after rebooting:

```
1 sudo cat /sys/kernel/debug/pinctrl/2430000.pinmux/pinmux-pins \
2     | grep -iE "spi1_mosi|spi1_miso|spi1_sck|spi1_cs0"
3 # expected output (all four lines):
4 # pin 133 (SPI1_SCK_PZ3): ... function spi1 group spi1_sck_pz3
5 # pin 134 (SPI1_MISO_PZ4): ... function spi1 group spi1_miso_pz4
6 # pin 135 (SPI1_MOSI_PZ5): ... function spi1 group spi1_mosi_pz5
7 # pin 136 (SPI1_CS0_PZ6): ... function spi1 group spi1_cs0_pz6
```

Reverting. Edit `/boot/extlinux/extlinux.conf` and change `DEFAULT AS5048A` to `DEFAULT primary`, then reboot. The timestamped backup created by the installer can also be restored verbatim.

3.3 Installing the Python spidev binding

The `spidev` package must be installed **system-wide** so that `sudo python3` (needed because `/dev/spidev*` is owned by `root/gpio`) can find it.

```
1 sudo pip3 install spidev # system-wide; do NOT use --user
2 sudo usermod -aG dialout,gpio,spi $USER # optional: grant non-root access
3 # log out and back in for group membership to take effect
```

Why not `--user`? Running `pip3 install --user spidev` installs the package under `$HOME/.local`, which is the calling user's home directory. When you then run `sudo python3`, Python uses `root`'s site-packages – a different path – and the import fails with `FAIL: spidev not installed`.

4 SPI bus mapping and the loopback test

4.1 The Tegra-name vs. Linux-number confusion

The Tegra silicon labels the four pads on the 40-pin header `spi1_mosi_pz5` etc., and the DTS function string is `"spi1"` – which suggests this is Linux SPI bus 1 (`/dev/spidev1.0`). It is not.

On the **Jetson Orin Nano Devkit (P3768)** the kernel probes the controllers in memory-map order:

Tegra block	Linux name	Device node
<code>spi@3210000</code>	<code>spi0</code>	<code>/dev/spidev0.0</code>
<code>spi@3230000</code>	<code>spi1</code>	<code>/dev/spidev1.0</code>

The 40-pin header pins 19/21/23/24 are wired to `spi@3210000`, so the correct device node is `/dev/spidev0.0` and the driver default is `AS5048AConfig(bus=0, device=0)`. The pad names with `"spi1"` in them reflect the *silicon block label*, not the Linux bus index.

On other Jetson carriers (e.g. AGX Orin) the same pads may be wired to `spi@3230000` = `/dev/spidev1.0`. Always run the loopback test (§4.2) on an unfamiliar board before hardcoding `bus=`.

4.2 Loopback test methodology

What it is. An SPI loopback test sends a known byte pattern out of MOSI and reads back whatever arrives on MISO. With a physical jumper wire connecting MOSI to MISO – and no chip on the bus to drive MISO independently – the received bytes must exactly match the transmitted bytes, proving that the SPI controller, the kernel driver, and the physical pin are all working end-to-end.

How to run it.

1. Disconnect the encoder from the header.
2. Short pin 19 (MOSI) to pin 21 (MISO) with a short jumper wire.
3. Run: `sudo python3 tests/encoder/00_spi_connectivity.py`

Stage 1 of the script probes both `/dev/spidev0.0` and `/dev/spidev1.0` without any manual input, transmitting the 8-byte test pattern `A5 5A 12 34 DE AD BE EF`.

Expected output with the overlay active and the jumper on pins 19/21:

```

1 Stage 1: SPI loopback test
2 /dev/spidev0.0 -- PASS tx=A5 5A 12 34 DE AD BE EF
3                   rx=A5 5A 12 34 DE AD BE EF
4 /dev/spidev1.0 -- FAIL rx all 0x00 (MISO floating)
5 -> SPI controller OK. Remove the jumper and reconnect the encoder.

```

spidev0.0 echoes perfectly because the MOSI pad is physically shorted back to the MISO pad. spidev1.0 reads all 0x00 because that controller is not wired to the 40-pin header on this board.

Failure modes and causes:

Bus	Result	Cause
spidev0.0	all 0x00	overlay not applied; SPI pads still tristated
spidev0.0	all 0xFF	MISO pulled high; jumper not making contact
spidev0.0	mismatch	wiring error or noisy jumper wire
both buses	all 0x00	not rebooted after <code>install.sh</code> , or <code>extlinux.conf</code> DEFAULT not set to AS5048A

After the loopback passes: remove the jumper, reconnect the encoder, and run Stage 2 of the script. A live chip (even without a magnet) returns non-zero words at all three test speeds (100 kHz / 500 kHz / 1 MHz); all-zero means a wiring or power problem.

5 SPI protocol

Every transaction is a 16-bit MSB-first frame. CS_n falls, 16 clocks shift, CS_n rises. At least 350 ns must elapse between two CS_n pulses (τ_{CSnH}); between frames the master typically idles the bus.

5.1 Frame formats

Command (master → AS5048A):

$$[\text{PAR}_{15}] [\text{RW}_{n14}] [\text{addr}_{13..0}]$$

where $\text{RW}_n = 1$ for read, 0 for write. Parity is **even** over bits 0–14: the parity bit is chosen so the total number of 1s in the 16-bit frame is even.

Response (AS5048A → master):

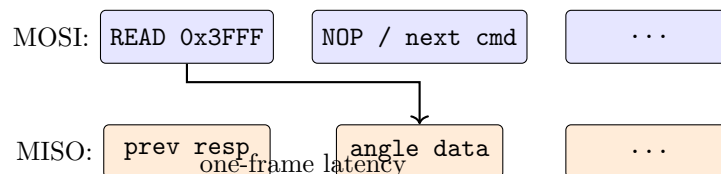
$$[\text{PAR}_{15}] [\text{EF}_{14}] [\text{data}_{13..0}]$$

EF = 1 means an error occurred in the *previous* host frame (parity / unknown command / wrong number of clocks). Read register 0x0001 (“Clear Error Flag”) to find out which.

Write data (master → AS5048A on the cycle following a write command):

$$[\text{PAR}_{15}] [R_{14} = 0] [\text{data}_{13..0}]$$

5.2 Read sequence (latency of one frame)



Send the READ command, then any next frame (typically NOP); the response containing the requested register sits in the MISO of that *second* frame.

5.3 Register map (relevant subset)

Address	Name	Notes
0x0000	NOP	response = 0; used as filler
0x0001	Clear Error Flag	read → clears EF, returns error bits
0x0003	Programming Control	Verify / Burn / Programming Enable
0x0016	OTP zero pos hi	bits ⟨13:6⟩ of zero offset
0x0017	OTP zero pos lo	bits ⟨5:0⟩ of zero offset
0x3FFD	Diag + AGC	OCF, COF, COMP_low, COMP_high, AGC⟨7:0⟩
0x3FFE	Magnitude	CORDIC magnitude (proxy for $ B_z $)
0x3FFF	Angle	zero-corrected 14-bit angle

5.4 Error register bits (response to 0x0001)

Bit	Name	Meaning
2	Parity Error	host parity wrong
1	Command Invalid	unknown address / RWn combination
0	Framing Error	not exactly 16 clocks between CSn edges

6 Driver layout in this repo

Mirror of the Damiao layout (see `docs/notes/dm_j4310_notes.tex`):

- `src/encoder_config.py` – single source of truth for SPI wiring. Exposes `AS5048AConfig(bus, device, max_hz, mode)` and `DEFAULT_ENCODER_CONFIG = AS5048AConfig(0, 0, 1000000, 1)`.
- `src/as5048a.py` – the `AS5048A` class. Provides `connect`, `disconnect`, `is_connected`, `__enter__`/`__exit__`, `read_angle_raw/_deg/_rad`, `read_magnitude`, `read_diagnostics`, `set_zero`, plus the low-level `read_register` / `write_register` primitives.
- `tests/encoder/` – numbered test scripts (see below).

6.1 Test scripts: summary

File	Purpose
<code>00_spi_connectivity.py</code>	SPI loopback + raw chip probe; run first on a new board
<code>01_read_angle.py</code>	50 Hz print loop of raw / deg / rad
<code>02_diagnostics.py</code>	AGC, magnitude and verdict on magnet placement
<code>03_set_zero.py</code>	latch current angle as zero (RAM only by default)
<code>04_zero_and_track.py</code>	zero current position, then track rotation with direction

6.2 Minimal usage

```
1 from encoder_config import DEFAULT_ENCODER_CONFIG
2 from as5048a import AS5048A
3
4 with AS5048A(DEFAULT_ENCODER_CONFIG) as enc:
5     print(enc.read_angle_deg()) # 0..360 degrees
6     print(enc.read_diagnostics()) # {'agc': 128, 'ocf': 1, ...}
7     enc.set_zero(burn_otp=False) # latch current pose as 0
```

7 Test scripts walkthrough – every SPI byte explained

This section is a line-by-line audit of the four scripts in `tests/encoder/`. For each script we list the exact bytes that travel on MOSI/MISO, decode every response, and explain what each field means in the firmware.

Notation used below.

- All SPI frames are 16-bit MSB-first. Python’s `spidev` operates on 8-bit lists, so a frame is sent as two bytes `[hi, lo]`.
- A **read** of address **A** requires two SPI frames: the first carries the READ command, the second carries any next command and *returns* the data in its MISO bytes. This is the “one-frame latency” from §4.2.
- Command word layout: `[PAR15 | RWn14 | addr13..0]`. PAR is even parity over bits 0..14, RWn=1 for read.
- Response word layout: `[PAR15 | EF14 | data13..0]`. EF=1 means an error occurred in the *previous* host frame.

7.1 00_spi_connectivity.py – wiring & chip check

Run this on a freshly-flashed Jetson with the overlay installed but before connecting any application code. It is divided into four stages and is the only script that bypasses the high-level driver and pokes `spidev` directly.

Stage 0: enumerate `/dev/spidev*`. A simple `glob.glob("/dev/spidev*")`. Expected on a Devkit: `[/dev/spidev0.0, /dev/spidev0.1, /dev/spidev1.0, /dev/spidev1.1]`. Missing nodes mean the overlay did not load (or DEFAULT AS5048A is not set in `extlinux.conf`).

Stage 1: SPI loopback.

```
1 test_pattern = [0xA5, 0x5A, 0x12, 0x34, 0xDE, 0xAD, 0xBE, 0xEF]
2 rx = spi.xfer2(list(test_pattern))
```

With pins 19 and 21 shorted, the SPI shift register clocks each MOSI byte out and immediately samples it back on MISO – so `rx == test_pattern` byte-for-byte.

Three possible outcomes per bus:

rx pattern	Meaning	Action
A5 5A 12 34 DE AD BE EF	PASS – controller works	remove jumper
00 00 00 00 ...	MISO floating	jumper not seated, or overlay missing
FF FF FF FF ...	MISO pulled high	wrong bus, no chip, no jumper
any other bytes	encoder is driving MISO	expected when chip is connected

The “MISMATCH” branch in the script prints a hint that this is *expected* when the encoder is connected: the chip drives MISO with its own response (typically the previous-frame data for whatever `0xA55A/0x1234/...` commands happen to decode as).

Stage 2: live-chip probe at three speeds. For each bus and each clock (100/500/1000 kHz) the script sends a single READ-DIAG command and collects the response:

```
1 spi.xfer2([0x7F, 0xFD]) # READ 0x3FFD
2 rx = spi.xfer2([0xFF, 0xFF]) # collect; payload is 0xFFFF (NOP'ish)
3 word = (rx[0] << 8) | rx[1]
```

Why 0x7FFD? Address 0x3FFD — RWn=1 (bit 14) = 0x7FFD. We compute its even parity:

$0x7FFD = 0111\ 1111\ 1111\ 1101_2$ has 14 one-bits in positions 0..14 $\Rightarrow$ parity = 0 $\Rightarrow$ PAR bit stays 0 $\Rightarrow$ frame = $0x7FFD$.

Interpreting the response. Three classes:

Response	Meaning
0x0000	MISO held low; no chip / wrong bus
0xFFFF	MISO pulled high; no chip
anything else	LIVE – chip is responding

Stage 3: parse the DIAG+AGC register. On the live bus the script issues:

1. [0x40, 0x01]: READ 0x0001 “Clear Error Flag” – wipes any sticky EF from previous transactions. (0x0001 | 0x4000 = 0x4001; even parity = 0.)
2. [0xFF, 0xFF]: dummy NOP that collects the error-flag response and discards it.
3. `time.sleep(0.1)`: 100 ms wait – the chip’s *Offset Compensation Finished* bit (OCF) only sets ~ 10 ms after VDD stabilises, and an OCF retry loop polls a further 500 ms if it is still 0.
4. Eight diag reads, then a re-poll loop if OCF=0.

Decoding the response word. Given a 16-bit word:

$$\underbrace{\text{PAR}_{15}}_{1 \text{ bit}} \underbrace{\text{EF}_{14}}_{1 \text{ bit}} \underbrace{\text{COMPHI}_{11}} \underbrace{\text{COMPLO}_{10}} \underbrace{\text{COF}_9} \underbrace{\text{OCF}_8} \underbrace{\text{AGC}_{7..0}}_{8 \text{ bits}}$$

Worked example. Field-test reading on this bench (no magnet, properly powered):

$$0x88FF = 1000\ 1000\ 1111\ 1111_2$$

Bits	Decoded	Meaning
15	PAR = 1	valid (frame has even number of 1-bits overall)
14	EF = 0	no error in the previous host frame
11	COMPHI = 1	magnetic field too weak (no magnet present)
10	COMPLO = 0	field not too strong
9	COF = 0	CORDIC has not overflowed
8	OCF = 0	offset compensation did <i>not</i> complete
7..0	AGC = $0xFF = 255$	gain saturated (no signal to amplify)

Interpretation: the chip is fully alive and the SPI framing is correct (EF=0, valid parity). AGC=255 + COMPHI=1 says “no magnetic field detected” – which is the expected state when the test is run without a magnet present. OCF=0 is the only ambiguous bit; in practice the AS5048A frequently leaves OCF=0 until a usable field appears, because the offset-compensation routine relies on the Hall sensors having a real signal to null. Placing the magnet typically flips OCF to 1 and drops AGC into the 100–200 range.

Stage 3 PASS criterion (after magnet placement):

$$\text{OCF} = 1, \text{COMPHI} = 0, \text{COMPLO} = 0, \text{AGC} \approx 128.$$

7.2 01_read_angle.py – continuous angle readout

The shortest script – a 50 Hz print loop that wraps the high-level driver:

```

1 with AS5048A(DEFAULT_ENCODER_CONFIG) as enc:
2     while True:
3         raw = enc.read_angle_raw()
4         deg = raw * (360.0 / 16384.0)
5         rad = raw * (6.283185307179586 / 16384.0)
6         print(f"\r{raw:5d} {deg:7.3f} {rad:7.4f}", end="", flush=True)
7         time.sleep(0.02)

```

What `read_angle_raw()` does internally.

1. Builds the READ-ANGLE command: $0x3FFF|0x4000 = 0x7FFF$. Its 16-bit population count is 15 (odd over bits 0..14, so parity bit = 1) $\Rightarrow$ final command = $0xFFFF$.
2. Sends frame 1: $[0xFF, 0xFF]$. (Coincidentally identical to a NOP-style filler.) The MISO bytes of this frame contain whatever the previous read had latched and are discarded.
3. Sends frame 2: another READ-NOP $0x4000$ command, which carries even parity = 0 $\Rightarrow$ bytes $[0x40, 0x00]$. The MISO of frame 2 now contains the angle.
4. Returns `word & 0x3FFF` – strips PAR and EF, keeps the 14-bit value.

Scaling. 16384 counts/rev ($=2^{14}$): $\text{deg} = \text{raw} \cdot 360/16384$, $\text{rad} = \text{raw} \cdot 2\pi/16384$.

Expected behaviour. With a magnet in place and `OCF=1`, the `raw` value smoothly walks through 0..16383 as the shaft turns. The 14-bit value wraps at 16383 $\rightarrow$ 0 in one LSB (0.022°). Without a magnet the angle register still returns a number, but it is dominated by Hall noise and jumps around – this is normal.

Parity-check note. `read_register()` in the driver currently calls `_even_parity(resp)` which XOR-folds only bits 0..14 of the response. Per the datasheet, the response is *valid* when the full 16-bit count is even (i.e. $\text{PAR} + \text{popcount of bits 0..14} \equiv 0$). A frame like $0x88FF$ ($\text{PAR}=1$, payload with 9 ones $\Rightarrow$ odd) is valid, but the current code treats odd payload as an error. If `01_read_angle.py` raises sporadic “AS5048A parity error” exceptions, fix `_even_parity` to XOR-fold all 16 bits and require the result to be 0.

7.3 `02_diagnostics.py` – AGC + magnet placement check

A 5 Hz print loop that prints the DIAG register plus the MAGNITUDE register ($0x3FFE$), then verbal verdict:

```
1 diag = enc.read_diagnostics()
2 mag = enc.read_magnitude()
3 if diag["comp_low"]: verdict = "FIELD TOO STRONG - increase air gap"
4 elif diag["comp_high"]: verdict = "FIELD TOO WEAK - decrease air gap"
5 elif diag["cof"]: verdict = "CORDIC OVERFLOW - check magnet alignment"
6 elif not diag["ocf"]: verdict = "offset comp not finished (wait...)"
7 else: verdict = "OK"
```

SPI traffic per loop iteration. Two reads, four frames:

#	Direction	Bytes (hex)	Meaning
1	MOSI	7F FD	READ $0x3FFD$
1	MISO	–	previous-frame data (discarded)
2	MOSI	C0 00	NOP ($0x0000$ as a READ = $0x4000$; $\text{PAR}=1$)
2	MISO	XX XX	DIAG word – see decode table above
3	MOSI	7F FE	READ $0x3FFE$
3	MISO	–	previous DIAG data (discarded)
4	MOSI	C0 00	NOP
4	MISO	YY YY	14-bit magnitude (CORDIC)

Magnitude register. A 14-bit unsigned value proportional to $\sqrt{B_x^2 + B_y^2}$. In a well-placed setup it sits in roughly 2000–4000; when the magnet is far away it drops near zero, when the chip is saturated it pegs near $0x3FFF$. It is informational only – the firmware does not use it to gate the angle output.

Tuning workflow:

1. Start with magnet 2–3 mm above the chip centred on the die.

2. Lower until `COMP_HI` flips from 1 to 0.
3. Continue lowering; if `COMP_LO` sets, you have over-shot – raise back up.
4. Aim for $AGC \approx 128$. An AGC of 64–192 is excellent; outside 32–224 means re-seat the magnet.

7.4 03_set_zero.py – programming the OTP zero

Programs registers 0x0016 / 0x0017 in RAM so the angle register returns 0 at the current mechanical pose. By default the change is volatile (lost on power cycle); pass `burn_otp=True` to fuse it permanently into OTP – this is irreversible and can be done only once per chip.

Algorithm (datasheet “Programming Sequence with Verification”).

1. Write 0 to `ZERO_HI` (0x0016) and `ZERO_LO` (0x0017). This zeroes any pre-existing offset so the next read returns the absolute mechanical angle.
2. Read the angle register 0x3FFF.
3. Write `angle<13:6>` to `ZERO_HI` and `angle<5:0>` to `ZERO_LO`. The chip subtracts this 14-bit value from every subsequent angle reading.

Write-frame layout reminder. A write costs three SPI frames:

Frame	MOSI bytes (e.g. write 0x05 to 0x0016)	Meaning
1	00 16	write-command ($RW_n=0$, addr 0x0016, parity 0)
2	80 05	data word 0x0005 + parity = 0x8005
3	C0 00	NOP (0x4000 — PAR) – flushes verify response

The verify byte returned in frame 3’s MISO is the value just written, echoed by the chip.

Example session.

```

1 Angle before zeroing : 187.404 deg
2 Latched offset : 8528 counts (187.404 deg)
3 Angle after zeroing : 0.022 deg # one-LSB residual is normal

```

OTP burn (only if `burn_otp=True`). After step 3:

1. Write 0x01 to `PROG_CONTROL` (0x0003) – Programming Enable.
2. Write 0x08 to `PROG_CONTROL` – Burn. Wait 20 ms.
3. Read the angle (should now be 0).
4. Write 0x40 to `PROG_CONTROL` – Verify (re-loads OTP).

Important: the OTP fuses can be programmed exactly once. Burn only when the mechanical mounting is finalised.

7.5 04_zero_and_track.py – zero then track rotation

This script combines the zeroing step with a live 50 Hz tracking loop. It is the primary script for verifying encoder behaviour on a mounted joint.

What it does:

1. Reads the current raw angle and prints it.
2. Calls `enc.set_zero(burn_otp=False)` to latch that angle as the new zero in RAM (volatile – lost on power cycle).
3. Enters a 50 Hz loop printing:

- `raw` – 14-bit count (0..16383).
- `deg` – angle relative to zero, wrapped to $(-180^\circ, +180^\circ]$.
- `rad` – same in radians.
- `dir` – CCW or CW vs. previous sample.
- `total_deg` – cumulative signed rotation (can exceed $\pm 360^\circ$ for multi-turn tracking).

4. On Ctrl+C: prints total rotation summary.

Signed-delta algorithm. Because the 14-bit counter wraps at $16383 \rightarrow 0$, a naive difference would give a spurious -16383 count when crossing the wrap boundary. The script uses the shortest-arc algorithm:

```
1 def signed_delta(a, b):
2     d = (b - a) % 16384 # always positive
3     if d > 8192: d -= 16384 # fold into (-8192, +8192]
4     return d # positive = CCW, negative = CW
```

This is correct for any single-sample step smaller than a half-turn ($8192 \text{ counts} = 180^\circ$), which is guaranteed at 50 Hz unless the shaft spins faster than $50 \times 180^\circ/\text{s} = 25 \text{ rev/s} = 1500 \text{ rpm}$.

Error-flag auto-retry. During a live run at 50 Hz over jumper wires, the chip’s EF (error flag) bit occasionally sets due to SPI bus noise triggering a spurious host-parity error in the chip. `read_register()` in `as5048a.py` clears EF and retries the read once before raising, so the loop runs uninterrupted. If EF persists on both attempts a real error is raised.

Example session (bench run, magnet on motor shaft):

```
1 Connected to /dev/spidev0.0 @ 1.0 MHz, mode 1
2
3 Current angle : 3624 counts ( 79.629 deg)
4 Zero latched : 7395 counts (162.488 deg offset)
5 Angle now : 327 counts ( 7.185 deg) (should be ~0)
6
7 Rotate the magnet. Ctrl+C to stop.
8
9   raw deg rad dir total_deg
10  -----
11 14683 -37.375 -0.6523 CW -764.978
```

In this run the magnet was rotated roughly -765° (about -2.1 turns CW) before the script was stopped.

Startup note. The first time `set_zero` writes 0 to 0x0016/0x0017 and reads back the angle, the result may be a few counts off zero due to the one-frame SPI latency – a residual of $\leq 3^\circ$ is normal and does not affect tracking accuracy.

8 Calibration: zero-position and magnet placement

8.1 Why zero matters

The chip’s encoder is absolute, but the mechanical zero of your joint will rarely coincide with the chip’s internal zero. Setting the **OTP zero registers** (0x0016/0x0017) makes the angle register return 0 at the desired mechanical pose, with no software offset maintained host-side.

8.2 Programming sequence (RAM, reversible)

This is what `enc.set_zero(burn_otp=False)` executes:

1. Write 0 to 0x0016 and 0x0017 – this clears any existing offset, so the next read returns the absolute mechanical angle.

2. Read the angle register 0x3FFF.
3. Split the 14-bit value: bits $\langle 13:6 \rangle$ to 0x0016, bits $\langle 5:0 \rangle$ to 0x0017.

The offset survives until power-cycle.

8.3 Permanent OTP burn (one-shot, irreversible)

After the steps above, set the Programming Enable bit (0x01) and then the Burn bit (0x08) in 0x0003, verify (0x40), and re-read. The OTP fuses can be burned **only once**. Use `burn_otp=True` in the driver *only* when you are sure of the mechanical mounting.

8.4 Magnet placement

The datasheet specifies a magnetic flux of 30–70 mT perpendicular to the die surface, at the chip centre. Practical checks:

- Use a 6–8 mm, ≥ 2.5 mm thick diametrically magnetised disc magnet (NdFeB or SmCo).
- Maintain 0.5–2.5 mm air gap from package surface.
- Centre the magnet over the package within $R_d \leq 0.25$ mm displacement (larger magnets allow up to 0.5 mm).
- Monitor the diagnostic flags: `comp_low=1` $\Rightarrow$ field too strong, `comp_high=1` $\Rightarrow$ field too weak. Target AGC near 128.

9 Common gotchas

1. **SPI mode is 1**, not 0. The datasheet wording “MOSI on falling CLK, MISO on rising CLK” = `CPOL=0`, `CPHA=1`. With mode 0 you read garbage.
2. **Two frames per read**. The response to a READ command lands in the *next* frame; the very first response after power-up is undefined. Discard it or send a dummy first read.
3. **Error flag is sticky**. Once EF is set, every subsequent response carries it until you issue Clear Error Flag (0x0001). The driver does this automatically on `connect()`.
4. **Parity is even, not odd**. XOR-fold bits 0..14, place the result in bit 15.
5. **Jumper-wire layouts**. Keep MISO/SCK short on a breadboard; ≥ 5 MHz over loose wires will give framing errors that surface as the EF bit. The driver default of 1 MHz is deliberately conservative.
6. **Permission to access /dev/spidev0.0**. The device node is owned by root and group `gpio` (mode 660). Use `sudo python3` or add yourself to the `gpio/spi` groups and log out/in.
7. **spidev must be installed system-wide for sudo**. `pip3 install --user spidev` installs under `$HOME/.local` which is invisible to root’s Python. Always use `sudo pip3 install spidev` (no `--user`) so that `sudo python3` can import it. See Section 3.3.
8. **Linux bus number $\neq$ Tegra pad label**. The pads are named `spi1.*pz*` in silicon and the DTS, but on the Orin Nano Devkit the 40-pin header routes to `spi@3210000 = /dev/spidev0.0`. Always verify with the loopback test (`00_spi_connectivity.py`) on any new board.

10 Useful references

- `docs/Encoder-AS5048.pdf` – the datasheet (v1-11, 2018-Jan-29).
- `src/as5048a.py` – this repo’s driver, used by `tests/encoder/`.
- *spidev* (<https://github.com/doceme/py-spidev>) – the Python SPI binding used here.
- *iotexpert/AS5048A-Python* – minimal Python implementation for Linux SBCs (good reference for parity logic).
- *RobTillaart/AS5048A* – Arduino C++ port, well-commented; useful sanity check on register access timing.
- ODrive firmware (<https://github.com/odriverobotics/ODrive>) contains a production-quality C driver for the closely related AS5047, which speaks the same 16-bit even-parity SPI frame.